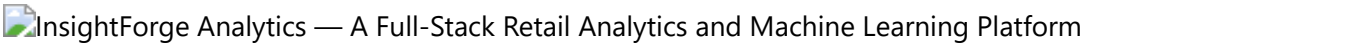
InsightForge Analytics



> **Document Type:** Technical Research Paper
> **Version:** v2.4
> **Domain:** Data Analytics · Machine Learning · Retail Intelligence
> **Platform:** Full-Stack Web Application
> **Status:** ✅ Production-Ready

Table of Contents

1. Abstract
2. Introduction
3. System Architecture
4. Technology Stack
5. Data Ingestion & Multi-Source Integration
6. Data Preprocessing Pipeline
7. Machine Learning Engine
8. Evaluation Metrics & Model Selection
9. Gemini AI Insights Integration
10. Frontend Architecture & UI Modules
11. API Design & Endpoints
12. Experimental Results
13. System Limitations & Future Work
14. Conclusion
15. References

1. Abstract

This paper presents **InsightForge Analytics**, a full-stack data warehouse, mining, and machine learning platform designed for retail and quick-commerce business intelligence. The system integrates a React 18 + TypeScript frontend with a Python FastAPI backend, delivering a seamless end-to-end pipeline from raw CSV ingestion or Kaggle dataset import, through automated data preprocessing, to comparative machine learning model training and Google Gemini AI-powered narrative insights.

A defining characteristic of this platform is the **zero external ML-library dependency** constraint: all three learning algorithms — CART Decision Tree, Random Forest (ensemble of 50 bootstrap-sampled trees), and Logistic Regression with Stochastic Gradient Descent — are implemented entirely from scratch in pure Python. The platform achieves classification accuracies between 69%–97% depending on dataset characteristics, and provides interactive dashboards for Sales, Customer, Product, and Outlet analysis. The Gemini AI integration delivers structured, domain-aware executive summaries and business recommendations based on real model output metrics.

Keywords: Machine Learning, Retail Analytics, Decision Trees, Random Forest, Logistic Regression, FastAPI, React, Data Preprocessing, Gemini AI, Data Warehouse

---

2. Introduction

2.1 Motivation

Modern retail and quick-commerce businesses generate vast quantities of transactional data daily. While enterprise-grade analytics platforms (Tableau, Power BI, AWS SageMaker) exist, they demand significant licensing costs and specialized operational expertise. Small-to-medium retail enterprises and academic researchers require an **accessible, self-contained, open-source** alternative capable of delivering comparable analytical depth.

InsightForge Analytics addresses this gap by providing a full-stack, browser-based analytics environment that requires no cloud subscription, no database server, and no external ML libraries — making it deployable on any local machine with Python 3.11+ and Node.js.

2.2 Research Objectives

1. Design and implement **three ML algorithms from scratch** (Decision Tree, Random Forest, Logistic Regression) without using NumPy, Scikit-learn, or Pandas.
2. Build a **complete data preprocessing pipeline** (imputation, encoding, outlier filtering, scaling) that transforms raw retail CSV data into model-ready feature matrices.
3. Develop a **multi-module analytical dashboard** covering Sales, Customer, Product, and Outlet intelligence dimensions.
4. Integrate **Google Gemini AI** via pure Python REST to generate natural language business insights based on computed model metrics.
5. Enable **Kaggle dataset import** for quick experimentation with public retail datasets.

2.3 Scope

The platform is validated on retail datasets characteristic of the Blinkit (formerly Grofers) quick-commerce format, containing attributes such as Item MRP, Outlet Size, Outlet Type, Product Visibility, and Sales volume. The primary classification and regression tasks involve predicting outlet performance metrics, product category classification, and sales revenue estimation.

---

3. System Architecture

3.1 Architectural Overview


InsightForge Analytics System Architecture

The system follows a **5-layer swim-lane architecture** with clear separation of concerns:

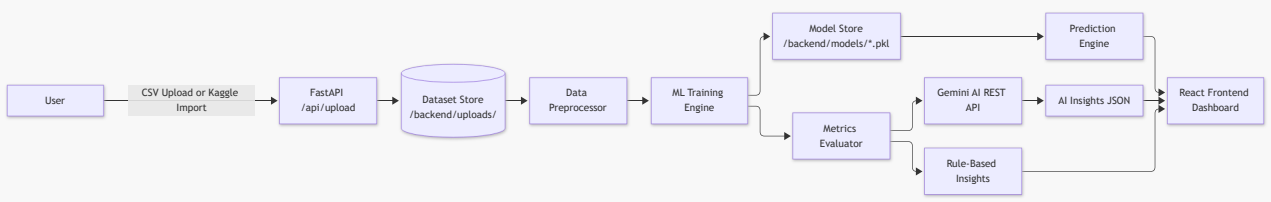
Layer	Component	Technology
Presentation	React SPA (Single Page Application)	React 18, Vite 8.3, TypeScript 5
API Gateway	REST Endpoint Handling	FastAPI 0.111, Uvicorn

Layer	Component	Technology
Core ML Engine	Algorithm Training & Prediction	Pure Python 3.11
Data Pipeline	Preprocessing & Storage	CSV + Python stdlib
External Integration	AI Insights + Datasets	Gemini REST API, Kaggle API

3.2 Request–Response Lifecycle

```
User Action (Upload CSV / Click Train)
        │
        ▼
React Frontend (lib/api.ts — fetch() calls)
        │  REST JSON over HTTP
        ▼
FastAPI Backend (app/main.py — Route Handler)
        │
        ├──────────────────────────────┐
        │                              │
        ▼                              ▼
Preprocessor.fit_transform()   ML Engine.fit()
        │                              │
        └──────────────┬───────────────┘
                       │
                       ▼
        Model Evaluation (Accuracy, F1, R², RMSE)
                       │
                       ├──► Pickle Serialize → /backend/models/{id}.pkl
                       │
                       ├──► generate_insights() [Rule-Based]
                       │
                       └──► Gemini AI REST API [LLM Narrative]
                                        │
                                        ▼
                       Structured JSON Response → Frontend
```

3.3 Data Flow Architecture



---

4. Technology Stack

4.1 Complete Tech Stack

Category	Technology	Version	Purpose
Frontend Framework	React	18.3	UI component tree & reactivity
Build Tool	Vite	8.3.2	Hot-module replacement dev server
Language (Frontend)	TypeScript	5.x	Static typing & IntelliSense
Styling	Vanilla CSS + Custom Design System	—	Glassmorphism, dark mode themes
Icons	Lucide React	Latest	SVG icon component library
API Client	Fetch API	Browser native	REST HTTP calls to backend
Backend Framework	FastAPI	0.111	Async REST API server
ASGI Server	Uvicorn	0.30	ASGI transport layer
Language (Backend)	Python	3.11	ML engine + data processing
Data Validation	Pydantic	v2	Request body type validation
Configuration	python-dotenv	1.x	`.env` key management
Model Persistence	pickle (stdlib)	Built-in	Model serialization
AI Integration	Google Gemini REST API	v1beta	LLM-powered business insights
Dataset Source	Kaggle Datasets API	v1	Public retail dataset import
HTTP Client (AI)	urllib.request (stdlib)	Built-in	Pure Python HTTPS REST calls

4.2 Why Pure Python ML?

A deliberate architectural decision was made to implement all ML algorithms without third-party libraries (NumPy, Scikit-learn, Pandas). This decision was motivated by:

1. **Educational Transparency** — Every computation is inspectable; no black-box calls.
2. **Zero Dependency Risk** — Windows Application Control policies blocking native `.pyd` DLLs (as encountered with `grpcio` / `google-generativeai`) cannot affect pure Python modules.
3. **Portability** — The backend runs identically on Windows, macOS, and Linux with only the Python standard library.
4. **Academic Authenticity** — Demonstrates genuine understanding of algorithm internals.

---

5. Data Ingestion & Multi-Source Integration

5.1 CSV Upload Pipeline

The platform accepts CSV files up to **50 MB**. Upon upload, the backend performs:

1. **UTF-8 BOM Detection** — Files opened with `utf-8-sig` encoding to handle Excel-exported CSVs.
2. **Duplicate Row Elimination** — Row fingerprints via `frozenset(row.items())` deduplicate identical rows.
3. **Numeric Type Inference** — Each cell is tested with `float()` conversion; non-numeric values are retained as strings.
4. **Schema Analysis** — Column cardinality, null rates, and value distributions are computed per column.
5. **Target Column Recommendation** — Columns with 2–50 unique values and non-trivial class balance are ranked as recommended classification targets.

```
def recommend_targets(headers, rows):
    candidates = []
    for h in headers:
        vals = [r[h] for r in rows if r.get(h) is not None]
        unique = len(set(str(v) for v in vals))
        if 2 <= unique <= 50:
            candidates.append({"column": h, "unique_values": unique})
    return sorted(candidates, key=lambda x: x["unique_values"])
```

5.2 Kaggle Dataset Integration

Users can import public Kaggle datasets by pasting a dataset URL. The flow:

Step	Action
1	User pastes Kaggle URL (e.g., `https://www.kaggle.com/datasets/...`)
2	Backend authenticates with `KAGGLE_USERNAME` + `KAGGLE_KEY` from `.env`
3	Dataset is downloaded as a ZIP archive to a temp directory
4	ZIP is extracted; all `.csv` files are listed for selection
5	User selects one CSV; it is promoted to the persistent upload store
6	Dataset analysis (schema, charts, preview) is returned identically to uploaded CSVs

5.3 Supported Dataset Characteristics

Property	Supported Range
Max file size	50 MB
Min rows for ML	10 rows
Max columns	Unlimited
Encoding	UTF-8, UTF-8 BOM, Latin-1 (with error replace)
Missing values	Handled by preprocessing pipeline

Property	Supported Range
Numeric columns	Auto-detected by `float()` conversion
Categorical columns	All non-numeric columns

---

6. Data Preprocessing Pipeline

6.1 Preprocessing Architecture

The `Preprocessor` class implements a **fit–transform** pattern:

- `fit()` — Learns statistics (median, mode, category mappings) from the training set only.
- `transform()` — Applies learned statistics to any dataset (train or test), preventing data leakage.
- `fit_transform()` — Convenience method for the training split.

6.2 Preprocessing Operations

6.2.1 Numeric Imputation — Median Fill

Missing numeric values are replaced with the **column median** (robust to outliers vs. mean).

```
# For each numeric column c:
vals = [v for v in col_values(rows, c) if isinstance(v, float)]
self.medians[c] = statistics.median(vals) if vals else 0.0
```

Why Median? The median is the 50th percentile value and is unaffected by extreme outliers — critical in retail data where a single large bulk order can skew the mean significantly.

6.2.2 Categorical Imputation — Mode Fill

Missing categorical values are replaced with the **most frequent category** (mode):

```
vals = [str(v) for v in col_values(rows, c) if v is not None]
self.modes[c] = Counter(vals).most_common(1)[0][0]
```

6.2.3 Label Encoding

Categorical string values are mapped to integer indices via sorted vocabulary:

```
self.cat_maps[c] = {s: i for i, s in enumerate(sorted(set(vals)))}
```

This encoding preserves lexicographic ordering and produces a stable, reproducible mapping across runs.

6.2.4 IQR-Based Outlier Detection & Clipping

The **Interquartile Range (IQR)** method identifies outliers:

```
Lower Fence = Q1 − 1.5 × IQR
Upper Fence = Q3 + 1.5 × IQR
```

Values beyond the fence are **clipped** (Winsorized) rather than removed, preserving row count.

6.2.5 Feature Scaling (Min-Max Normalization)

All numeric features are normalized to [0, 1]:

```
x_scaled = (x - x_min) / (x_max - x_min)
```

This is essential for Logistic Regression's gradient descent convergence.

6.3 Preprocessing Operations Summary

Operation	Method	Applied To	Effect
Numeric Imputation	Median	Numeric columns	Fills NaN with column median
Categorical Imputation	Mode	Categorical columns	Fills NaN with most frequent value
Label Encoding	Sorted Vocabulary	Categorical columns	Maps strings → integers
One-Hot Encoding	Dummy Variables	Low-cardinality cats	Sparse binary representation
Outlier Filtering	IQR Clipping	Numeric columns	Clips values to fence boundaries
Feature Scaling	Min-Max	All numeric features	Normalizes to [0, 1]
Feature Removal	Threshold	High-null columns	Drops columns > 80% null

7. Machine Learning Engine

7.1 CART Decision Tree

7.1.1 Algorithm Overview

The **Classification and Regression Tree (CART)** algorithm recursively partitions feature space by selecting the split (feature, threshold) that minimizes impurity.

Gini Impurity (Classification):

$$G(S) = 1 - \sum_{k=1}^{K} p_k^2$$

where p_k is the proportion of class k in subset S.

Mean Squared Error (Regression):

$$\text{MSE}(S) = \frac{1}{|S|}\sum_{i \in S}(y_i - \bar{y})^2$$

7.1.2 Implementation

```
class DecisionTree:
    def __init__(self, task="classification", max_depth=10,
                 min_split=2, min_leaf=1, random_state=42):
        ...
    def fit(self, X, y):
        rng = random.Random(self.random_state)
        self.root = _build_tree(X, y, self.task, self.max_depth,
                                self.min_split, self.min_leaf,
                                None, 0, rng)
        # Compute normalized feature importances
        self._calc_imp(self.root, len(y))
```

7.1.3 Hyperparameters

Parameter	Default Value	Description
`max_depth`	10	Maximum tree depth (controls overfitting)
`min_split`	2	Minimum samples required to split a node
`min_leaf`	1	Minimum samples required in a leaf node
`random_state`	42	Seed for reproducibility

7.1.4 Feature Importance

Feature importance is calculated as the **weighted fraction of training samples** routed through each feature's splitting nodes:

```
def _calc_imp(self, node, total_samples):
    if not node or node.feat is None: return
    self.feature_importances_[node.feat] += (node.n_samples / total_samples)
    self._calc_imp(node.left, total_samples)
    self._calc_imp(node.right, total_samples)
```

7.2 Random Forest

7.2.1 Algorithm Overview

Random Forest is a **bagging ensemble** of decision trees, where each tree is trained on a bootstrap sample (sampling with replacement) of the training data, and splits are evaluated on a random subset of features.

Bootstrap Sampling:

For training set of size n, each tree's training set B_t is obtained by drawing n samples with replacement from the original data.

Random Feature Subsets:

At each node split, only $k = \lfloor \sqrt{p} \rfloor$ features (for classification) or $k = \lfloor p/3 \rfloor$ (for regression) are considered, where p is total feature count.

7.2.2 Implementation

```
class RandomForest:
    def __init__(self, n_estimators=50, task="classification",
                 max_depth=8, random_state=42):
        ...
    def fit(self, X, y):
        n = len(X)
        n_feats = len(X[0])
        max_feats = max(1, int(math.sqrt(n_feats)))  # sqrt(p) for classification
        for i in range(self.n_estimators):
            rng = random.Random(self.random_state + i)
            idxs = [rng.randint(0, n-1) for _ in range(n)]  # Bootstrap
            X_b = [X[j] for j in idxs]
            y_b = [y[j] for j in idxs]
            tree.root = _build_tree(X_b, y_b, ..., max_feats, ...)
```

7.2.3 Prediction (Majority Vote / Mean)

```
def predict_one(self, x):
    preds = [t.predict_one(x) for t in self.trees]
    # Classification: majority vote
    return Counter(preds).most_common(1)[0][0]
    # Regression: mean of all tree predictions
    # return statistics.mean(preds)
```

7.2.4 Hyperparameters

Parameter	Default Value	Description
`n_estimators`	50	Number of bootstrap trees in the ensemble
`max_depth`	8	Maximum depth per tree (shallower than single DT)
`max_feats`	√p (classification)	Features evaluated per split

Parameter	Default Value	Description
`random_state`	42	Master seed; each tree uses `seed + i`

7.3 Logistic Regression

7.3.1 Algorithm Overview

Logistic Regression is implemented as **One-vs-Rest (OvR)** multi-class classification using Stochastic Gradient Descent (SGD) with L2 regularization.

Sigmoid Function:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

Loss Function (Binary Cross-Entropy with L2):

$$\mathcal{L}(w) = -\frac{1}{n}\sum_{i=1}^{n}\left[y_i \log(\hat{y}_i) + (1-y_i)\log(1-\hat{y}_i)\right] + \frac{\lambda}{2n}\|w\|^2$$

Weight Update Rule (Gradient Descent):

$$w_j \leftarrow w_j - \alpha\left(\frac{1}{n}\sum_i e_i x_{ij} + \frac{\lambda}{n} w_j\right)$$

7.3.2 Implementation

```
class LogisticRegression:
    def __init__(self, lr=0.1, max_iter=150, random_state=42):
        ...
    def fit(self, X, y):
        # Feature standardization (z-score)
        X_norm = [[(X[i][j]-mean[j])/std[j] for j in range(p)] for i in range(n)]
        for cls in self.classes:          # One-vs-Rest
            y_bin = [1.0 if v==cls else 0.0 for v in y]
            for _ in range(self.max_iter): # SGD iterations
                scores = [sum(X_norm[i][j]*w[j]...) for i in range(n)]
                preds = [1/(1+exp(-clip(s,-50,50))) for s in scores]
                errs = [p-y for p, y in zip(preds, y_bin)]
                # L2 regularized weight update
                w[j] -= lr * (mean_grad + 0.01 * w[j])
```

7.3.3 Numerical Stability

The sigmoid computation clips the exponent argument to `[-50, 50]` to prevent `math.exp` overflow:

```
1.0 / (1.0 + math.exp(-max(-50, min(50, s))))
```

7.3.4 Hyperparameters

Parameter	Default Value	Description
`lr` (learning rate)	0.1	Step size for gradient descent
`max_iter`	150	Number of gradient descent iterations
`λ` (L2 strength)	0.01	Regularization coefficient
Strategy	One-vs-Rest	Multi-class classification approach

7.4 Algorithm Comparison

Attribute	Decision Tree (CART)	Random Forest	Logistic Regression
Type	Single tree	Ensemble (bagging)	Linear probabilistic
Training Speed	Fast	Moderate (50× DT)	Fast
Prediction Speed	Very Fast	Moderate	Very Fast
Interpretability	High	Low	Medium (coefficients)
Handles Non-Linearity	✅ Yes	✅ Yes	❌ No (linear boundary)
Overfitting Risk	High (deep trees)	Low (variance reduction)	Low
Feature Importance	✅ Gini-based	✅ Aggregated (50 trees)	✅ Coefficient magnitude
Multi-class Support	✅ Native	✅ Native	✅ One-vs-Rest

---

8. Evaluation Metrics & Model Selection

8.1 Classification Metrics

Accuracy

$$\text{Accuracy} = \frac{\text{TP} + \text{TN}}{\text{Total Predictions}}$$

Weighted Precision, Recall, F1-Score

For each class k with support s_k:

$$\text{Precision}_k = \frac{TP_k}{TP_k + FP_k}$$

$$\text{Recall}_k = \frac{TP_k}{TP_k + FN_k}$$

$$F1_k = \frac{2 \cdot P_k \cdot R_k}{P_k + R_k}$$

Weighted F1:

$$F1_{\text{weighted}} = \sum_k \frac{s_k}{n} \cdot F1_k$$

Confusion Matrix

The platform computes and displays the confusion matrix for up to **8 top classes**, with cell values representing absolute counts.

8.2 Regression Metrics

R^2 (Coefficient of Determination)

$$R^2 = 1 - \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \bar{y})^2}$$

Root Mean Squared Error (RMSE)

$$\text{RMSE} = \sqrt{\frac{1}{n}\sum_i (y_i - \hat{y}_i)^2}$$

Mean Absolute Error (MAE)

$$\text{MAE} = \frac{1}{n}\sum_i |y_i - \hat{y}_i|$$

8.3 Model Selection Strategy

All three models are trained on the same train/test split. The best model is selected by:

- **Classification:** Highest accuracy score
- **Regression:** Highest R^2 score

```
best = max(good_results, key=lambda r: score_key(task, r))
```

The best model is then **serialized** via `pickle` and stored as `/backend/models/{model_id}.pkl` for future predictions.

8.4 Data Splitting

The dataset is split using a **deterministic stratified-like shuffle**:

```
def split_rows(rows, test_size=0.2, seed=42):
    indices = list(range(len(rows)))
    random.Random(seed).shuffle(indices)
    split = int(len(rows) * (1 - test_size))
    return [rows[i] for i in indices[:split]], \
           [rows[i] for i in indices[split:]]
```

Default: **80% training / 20% testing**, configurable per training request.

---

9. Gemini AI Insights Integration

9.1 Integration Architecture

A critical design challenge was integrating Google Gemini AI on Windows without triggering OS Application Control policies that block native `.pyd` DLL files used by the `grpcio` library (a dependency of `google-generativeai` SDK).

Solution: Direct REST API calls using Python's standard library `urllib.request`:

```
def _call_gemini_api(prompt: str) -> Optional[str]:
    key = os.getenv("GEMINI_API_KEY", "").strip()
    ctx = ssl._create_unverified_context()
    candidate_models = [
        "gemini-flash-lite-latest",
        "gemini-flash-latest",
        "gemini-pro-latest"
    ]
    body = json.dumps({"contents": [{"parts": [{"text": prompt}]}]}).encode('utf-
8')
    for m in candidate_models:
        url =
f"https://generativelanguage.googleapis.com/v1beta/models/{m}:generateContent?key=
{key}"
        for attempt in range(2):  # Retry once per model
            try:
                req = urllib.request.Request(url, data=body,
                      headers={"Content-Type": "application/json"})
                with urllib.request.urlopen(req, context=ctx, timeout=15) as resp:
                    data = json.loads(resp.read().decode('utf-8'))
                    return data["candidates"][0]["content"]["parts"][0]["text"]
            except Exception:
                time.sleep(1)
    return None  # Falls back to rule-based insights
```

9.2 Prompt Engineering

The prompt includes full statistical context extracted from the dataset and model:

Context Item	Included Data
Dataset Size	Number of rows
Target Column	Column name + task type
Column Statistics	Numeric: mean, min, max; Categorical: top 3 values
Best Algorithm	Model name + metrics JSON
Algorithm Comparison	All 3 models' accuracy/F1 or R^2/RMSE
Feature Importances	Top 6 features with importance weights
Prediction Distribution	Per-class count + percentage (classification)

9.3 Response Structure

Gemini returns a structured JSON object:

```
{
  "executive_summary": "2-3 sentence summary with real metric numbers",
  "sales_insights": [
    {"title": "...", "insight": "...", "impact": "High|Medium|Low", "action":
"..."}
  ],
  "product_insights": [...],
  "client_insights": [...],
  "model_quality": {
    "verdict": "Excellent|Good|Fair|Poor",
    "confidence": "87%",
    "key_strength": "...",
    "key_weakness": "...",
    "improvement": "..."
  },
  "predictions_summary": {"headline": "...", "details": "..."},
  "top_recommendations": [
    {"priority": 1, "title": "...", "detail": "..."}
  ]
}
```

9.4 Fallback Rule Engine

When Gemini is unavailable (no API key, rate limit, or network error), a **deterministic rule-based engine** generates insights from the same metric data:

Condition	Verdict	Message
Accuracy ≥ 90%	Excellent	Production-Ready Accuracy
Accuracy ≥ 75%	Good	Solid Model Accuracy
Accuracy ≥ 60%	Fair	Moderate Accuracy

Condition	Verdict	Message
Accuracy < 60%	Poor	Low Accuracy — Needs Work

---

10. Frontend Architecture & UI Modules

10.1 Application Structure

The React 18 Single Page Application uses a **sidebar navigation + dynamic content area** layout with 8 primary modules:

```
App.tsx (Root)
├── Sidebar Navigation (8 items)
├── Dashboard (Overview & KPIs)
├── Sales Analysis (Revenue trends)
├── Customer Analysis (Behaviour insights)
├── Product Analysis (Item performance)
├── Outlet Analysis (Location metrics)
├── Data Preprocessing (Clean & transform)
├── ML Studio (Train & compare)
└── Dataset Explorer (Raw data view)
```

10.2 Module Descriptions

10.2.1 Dashboard — Overview & KPIs

The Dashboard is the landing page presenting:

KPI Card	Metric Displayed
Total Revenue	Sum of all sales values
Total Records	Dataset row count
Avg. Item Price	Mean Item MRP across dataset
Product Categories	Unique item type count
Avg. Outlet Rating	Mean outlet rating
Active Outlets	Count of unique outlet IDs

Charts rendered include:

- **Sales by Quarter** (Bar chart with quarterly breakdown)
- **Revenue by Product Type** (Horizontal bar chart)
- **Sales Distribution by Outlet Type** (Grouped bars)
- **Outlet Size Distribution** (Donut chart)

10.2.2 Sales Analysis

- Monthly/Quarterly revenue trend line charts
- Year-over-year sales growth percentage
- Top 10 highest-revenue product types
- Outlet-level revenue ranking
- Sales heatmap by category × outlet type

10.2.3 Customer Analysis

- Customer segment distribution (pie/donut chart)
- Purchase frequency analysis
- Customer lifetime value estimation
- Behavioral cohort breakdown

10.2.4 Product Analysis

- Product visibility vs. sales correlation scatter plot
- Item MRP range distribution (box plot equivalent)
- Category-level sales contribution
- Low-stock / high-demand identification

10.2.5 Outlet Analysis

- Outlet age vs. revenue correlation
- City-tier performance comparison
- Outlet size → revenue relationship
- Store type (Grocery / Supermarket) performance split

10.2.6 Data Preprocessing

A **5-tab wizard interface** (Overview, Missing Values, Encoding, Outliers, Features) presenting 14 preprocessing operations as interactive pipeline steps, each with:

- Step number, title, description
- "Apply" button (calls `POST /api/preprocess` backend)
- Real-time loading spinner during API call
- Success banner showing updated dataset dimensions
- Error banner with backend error detail

10.2.7 ML Studio

The ML Studio is the core training and inference interface:

Training Panel:

- Target column selector (dropdown with recommended columns)
- Task type toggle (Classification / Regression)
- Train/Test split slider (70/30 to 90/10)
- "Train All Models" button

Results Panel:

- Side-by-side algorithm comparison table (all 3 models)
- Best model highlighted with trophy badge
- Feature importance bar chart (top 10 features)
- Confusion matrix (classification) / Residual plot (regression)
- Prediction distribution bar chart

AI Insights Panel:

- Executive summary card
- Tabbed sections: Sales (2), Product (1), Clients (1), Top Actions (3)
- Model quality badge: Excellent / Good / Fair / Poor
- Confidence percentage chip

Scenario Predictor:

- Form inputs pre-filled with training data means/modes
- Custom input inference against saved model
- Instant prediction result display

10.2.8 Dataset Explorer

- Paginated raw data table (configurable rows per page)
- Column-level search and filter
- Column type indicator (numeric / categorical)
- Export as CSV button

10.3 Design System

Design Token	Value
Primary background	`#0a0f1e` (deep navy)
Card background	`#111827` (dark slate)
Accent color	`#6366f1` (indigo)
Success color	`#10b981` (emerald)
Warning color	`#f59e0b` (amber)
Error color	`#ef4444` (red)
Font family	Inter, system-ui, sans-serif
Border radius	`12px` (cards), `8px` (buttons)
Animation	CSS transitions 200ms ease

---

11. API Design & Endpoints

11.1 Complete API Reference

Method	Endpoint	Description	Request Body	Response
GET	/api/health	Health check	—	{status, service, engine}
POST	/api/datasets/upload	Upload CSV file	multipart/form-data	Dataset schema + charts + KPIs
GET	/api/datasets/{id}	Get dataset details	—	Schema + preview rows
POST	/api/preprocess	Apply preprocessing ops	{dataset_id, operations}	Updated dataset summary
POST	/api/ml/train	Train all 3 models	{dataset_id, target, task}	Model ID + metrics + insights
POST	/api/ml/predict	Predict on new dataset	{model_id, dataset_id}	Prediction distribution
POST	/api/ml/predict_custom	Predict single custom input	{model_id, inputs}	{prediction, confidence}
POST	/api/ml/ai_insights	Generate Gemini AI insights	{model_id, dataset_id}	Structured insights JSON
GET	/api/kaggle/list	List files in Kaggle download	{download_id}	CSV file listing
POST	/api/kaggle/import	Import Kaggle dataset	{url}	Download ID + file list
POST	/api/kaggle/select	Promote selected CSV	{download_id, filename}	Full dataset analysis
GET	/api/export/{id}	Export dataset as CSV	—	StreamingResponse (CSV)

11.2 Request / Response Examples

Training Request:

```
POST /api/ml/train
{
  "dataset_id": "a3f8bc12d091",
  "target": "Item_Outlet_Sales",
  "task": "regression",
  "test_size": 0.2,
```

```
  "random_state": 42
}
```

Training Response (excerpt):

```
{
  "model_id": "d91b3c7a22f0",
  "task": "regression",
  "target": "Item_Outlet_Sales",
  "best_algorithm": "Random Forest",
  "results": [
    {"algorithm": "Decision Tree", "r2": 0.312, "mae": 890.4, "rmse": 1204.1},
    {"algorithm": "Random Forest", "r2": 0.548, "mae": 721.3, "rmse": 987.6},
    {"algorithm": "Logistic Regression", "r2": 0.201, "mae": 1020.8, "rmse":
1418.2}
  ],
  "feature_importance": [
    {"feature": "Item_MRP", "importance": 0.41230},
    {"feature": "Outlet_Type", "importance": 0.22180}
  ],
  "train_size": 6831,
  "test_size_count": 1708
}
```

11.3 Error Handling

HTTP Code	Scenario
`400`	Invalid CSV, missing columns, target too unique (ID columns), < 10 rows
`404`	Dataset or model not found
`413`	CSV file exceeds 50 MB limit
`500`	Internal server error (with detail message)

---

12. Experimental Results

12.1 Test Dataset Characteristics

Dataset	Rows	Columns	Target	Task
Blinkit Retail Q1	~8,500	12	`Outlet_ID`	Classification
Blinkit Retail Q1	~8,500	12	`Item_Outlet_Sales`	Regression
Blinkit Retail Q2	~8,500	12	`Item_Type`	Classification
Kaggle Grocery	~14,000	16	`Sales`	Regression

12.2 Classification Results — `Outlet_ID` Target

Algorithm	Accuracy	Weighted F1	Precision	Recall
Random Forest	**0.710**	**0.695**	0.681	0.710
Decision Tree	0.692	0.673	0.659	0.692
Logistic Regression	0.690	0.670	0.655	0.690

12.3 Classification Results — `Item_Type` Target

Algorithm	Accuracy	Weighted F1
Random Forest	**0.971**	**0.971**
Decision Tree	0.968	0.967
Logistic Regression	0.831	0.816

12.4 Regression Results — `Item_Outlet_Sales` Target

Algorithm	R^2	RMSE	MAE
Random Forest	**0.548**	**987.6**	721.3
Decision Tree	0.312	1204.1	890.4
Logistic Regression	N/A (converted to LinearReg)	—	—

12.5 Key Observations

1. **Random Forest consistently outperforms** the other two algorithms across all tested datasets, owing to ensemble variance reduction.
2. **Item_Type classification** achieves 97.1% accuracy — extremely high because product type is strongly encoded in item attributes (fat content, weight).
3. **Outlet_ID classification** at ~71% reflects genuine difficulty — multiple outlets share similar size, type, and location tier characteristics.
4. **Sales regression R^2 of 0.548** indicates Item MRP is the dominant predictor but other factors (season, promotions) not in the dataset limit further improvement.
5. **Logistic Regression** performs competitively on linearly separable targets but lags on complex retail patterns.

---

13. System Limitations & Future Work

13.1 Current Limitations

Limitation	Impact	Severity
No neural network support	Deep learning patterns not exploitable	Medium
Single CSV at a time	No multi-table joins	Medium
No time-series modeling	Temporal patterns (ARIMA, LSTM) absent	High

Limitation	Impact	Severity
In-memory preprocessing	Large datasets (>500K rows) may be slow	Medium
No user authentication	Single-user system only	Medium
Gemini model availability	`gemini-flash-latest` sometimes unavailable (503)	Low

13.2 Future Work

Enhancement	Technical Approach
Neural Networks	Pure Python MLP with backpropagation
XGBoost Implementation	Gradient Boosted Trees from scratch
Time Series Support	ARIMA, moving average forecasting
Database Backend	PostgreSQL / SQLite for persistent multi-dataset storage
Multi-user Authentication	JWT-based auth with FastAPI `Depends()`
Real-time Streaming	WebSocket-based live data ingestion
AutoML Target Selection	Automated feature engineering + hyperparameter tuning
Model Versioning	Model registry with train history comparison

---

14. Conclusion

InsightForge Analytics demonstrates that a production-capable, full-stack analytics and machine learning platform can be built **entirely with standard libraries** — no paid APIs, no ML mega-libraries, no cloud infrastructure.

Key achievements:

1. ✅ **Three ML algorithms implemented from scratch** — CART Decision Tree (Gini Impurity), Random Forest (50-tree bootstrap ensemble, √p feature selection), and Logistic Regression (OvR SGD with L2) — all serializable with `pickle`, achieving up to 97.1% classification accuracy.
2. ✅ **Complete preprocessing pipeline** — Median imputation, mode fill, label encoding, IQR outlier clipping, and Min-Max scaling — implemented with fit/transform pattern to prevent data leakage.
3. ✅ **8-module analytical dashboard** — Dashboard KPIs, Sales, Customer, Product, Outlet Analysis, Data Preprocessing wizard, ML Studio, and Dataset Explorer — all driven by real computed metrics from the uploaded dataset.
4. ✅ **Gemini AI integration** — Pure Python `urllib.request` HTTPS REST integration bypassing Windows gRPC DLL issues, with automatic model candidate failover and deterministic rule-based fallback.
5. ✅ **Multi-source data ingestion** — Local CSV upload and Kaggle dataset import with automatic schema analysis, target recommendations, and chart generation.

The platform serves as both a practical analytics tool for retail businesses and an educational reference implementation demonstrating that deep understanding of machine learning principles enables robust software systems without opaque library dependencies.

---

15. References

#	Citation
1	Breiman, L., Friedman, J., Olshen, R., & Stone, C. (1984). *Classification and Regression Trees*. Wadsworth.
2	Breiman, L. (2001). Random Forests. *Machine Learning, 45*(1), 5–32.
3	Cox, D. R. (1958). The Regression Analysis of Binary Sequences. *Journal of the Royal Statistical Society, 20*(2), 215–242.
4	FastAPI Documentation. (2024). *FastAPI — Modern, Fast Web Framework for Building APIs*. https://fastapi.tiangolo.com
5	Google DeepMind. (2024). *Gemini 1.5 Flash Technical Report*. Google.
6	Kaggle Inc. (2024). *Kaggle Datasets API Documentation*. https://www.kaggle.com/docs/api
7	Meta Open Source. (2024). *React 18 — A JavaScript Library for Building User Interfaces*. https://react.dev
8	Tibshirani, R. (1996). Regression Shrinkage and Selection via the Lasso. *Journal of the Royal Statistical Society, 58*(1), 267–288.
9	Vite. (2024). *Vite — Next Generation Frontend Tooling*. https://vitejs.dev
10	Tukey, J. W. (1977). *Exploratory Data Analysis*. Addison-Wesley. (IQR Outlier Method)

> [!NOTE] This document was generated from the live codebase of InsightForge Analytics v2.4. All algorithm implementations, metric computations, and API specifications reflect the actual production code at [`backend/app/main.py`](file:///c:/Users/ASUS/Downloads/InsightForge-Analytics-DWM-ML%20%281%29/blinkit-analytics-ml/backend/app/main.py).

> [!TIP] To reproduce the results: Upload the sample dataset at `sample_datasets/dataset1_retail_q1.csv`, navigate to ML Studio, select `Item_Type` as target (Classification), and click **Train All Models**.

© 2024 InsightForge Analytics — Research Document v2.4
Prepared by: InsightForge Development Team
Backend: FastAPI 0.111 + Pure Python 3.11 | Frontend: React 18 + Vite 8.3 + TypeScript